Mawlana Bhashani Science and Technology University
Department of Information and Communication Technology

Software Engineering and Project Management

Project Proposal

FloodGuard: IoT-Based Flood Early Warning System for Riverside Villages

---

Submitted by: Jubayer Ahmed
ID: IT24027
Session: 2023–2024
Level: 3rd Year, 1st Semester

Submitted to: Professor Dr. Md. Ziaur Rahman
Department of ICT, MBSTU

Date: October 27, 2026

Contents

Abstract 3

1 Introduction 3

2 Problem Statement 3

3 Objectives 4

3.1 General objective . . . 4

3.2 Specific objectives . . . 4

4 Scope and Limitations 4

4.1 In scope . . . 4

4.2 Out of scope . . . 4

4.3 Limitations . . . 4

5 Literature Review and Existing Systems 4

6 SWOT Analysis 5

7 Requirements Specification 5

7.1 Functional requirements . . . 5

7.2 Non-functional requirements . . . 6

7.3 Input and output . . . 6

8 Feasibility Study 6

9 Risk Analysis 7

10 Cost and Resource Estimation 7

11 Software Process Model: Incremental Model 7

11.1 Reason for selection . . . 7

11.2 Increments . . . 8

11.3 Weekly schedule . . . 8

11.4 Milestones and deliverables . . . 8

11.5 Gantt chart . . . 9

12 System Architecture and Technology Stack 9

13 UML Diagrams 10

13.1 Data Flow Diagram (Level 0 and Level 1) . . . 10

13.2 Use Case Diagram 11
13.3 Activity Diagram 12
13.4 Entity Relationship Diagram 13
13.5 Class Diagram 14
13.6 Object Diagram 15

14 Simple Interface Implementation Plan **15**

15 Testing Plan **16**

16 Conclusion and Future Work **16**

Abstract

Bangladesh faces recurrent floods, and riverside communities along the Jamuna, including areas of Tangail, often receive warnings late or in a form that does not reach households. This proposal presents **FloodGuard**, a low-cost Internet of Things (IoT) system that measures river water level with an ESP32 microcontroller and an ultrasonic sensor, transmits the readings over WiFi (HTTP) to a Java Servlet server, classifies the situation as *Safe*, *Warning* or *Danger* against a configurable danger level, and notifies people through a web dashboard, local LED/buzzer indicators and Telegram messages. The project follows the Incremental process model over 12 weeks. The proposal contains the problem analysis, requirements, SWOT, feasibility, risk and cost analysis, schedule with Gantt chart, six UML diagrams, and a simple web interface implemented with more than five classes from the class diagram.

Keywords: IoT, ESP32, flood early warning, Java Servlet, ultrasonic sensor, Incremental model, UML.

1 Introduction

Floods are among the most frequent natural disasters in Bangladesh. The Brahmaputra–Jamuna river system regularly rises above its danger level during the monsoon, affecting districts such as Kurigram, Gaibandha, Jamalpur, Sirajganj and Tangail. The Flood Forecasting and Warning Centre (FFWC) of the Bangladesh Water Development Board (BWDB) publishes daily bulletins for monitored stations, but these are station-level and national-scale products. Village-level, automatic and immediate alerts remain limited.

Advances in low-cost microcontrollers (e.g. ESP32), cheap sensors and free software make it feasible to build a local monitoring and alerting node. FloodGuard is a prototype of such a node and the software that receives, analyses and communicates its data.

2 Problem Statement

- Water level monitoring at village scale is mostly manual and irregular.
- Official bulletins are periodic and do not directly reach every household.
- Residents and volunteers have no automatic, threshold-based alert tied to their nearest river point.
- Existing solutions are costly or require specialised infrastructure, which is difficult for a student or small community project.

Proposed solution: an IoT-based, low-cost system that continuously measures water level, evaluates it against the danger level, records history and issues staged alerts (Safe/Warning/Danger).

3 Objectives

3.1 General objective

To design and develop a low-cost IoT-based early warning prototype that monitors river water level and issues timely flood alerts.

3.2 Specific objectives

1. To measure water level using an ESP32 and an ultrasonic sensor (HC-SR04 / JSN-SR04T).
2. To transmit readings to a Java Servlet server over WiFi using HTTP.
3. To classify status as Safe ($<80\%$), Warning (80–100%) and Danger ($\geq 100\%$) of the danger level.
4. To display live status, graph and alert history on a web dashboard.
5. To generate local alarms (LED, buzzer) and notifications (console SMS simulation, Telegram).
6. To apply software engineering practice: Incremental model, UML modelling, testing and documentation.

4 Scope and Limitations

4.1 In scope

Single-river-point monitoring node; server-side processing; web dashboard; alert generation and history; user roles (Admin, Volunteer, Resident); simulated and real sensor input; UML design; LaTeX documentation.

4.2 Out of scope

Hydrological forecasting/machine learning; nationwide deployment; real SMS-gateway billing integration; mobile application; weatherproof field enclosure and solar power design (discussed as future work).

4.3 Limitations

The prototype is demonstrated by emulating water level (sensor distance variation in Wokwi or on a bench) and does not claim live river measurement. FloodGuard is a decision-support aid and does not replace official FFWC forecasts.

5 Literature Review and Existing Systems

- **FFWC / BWDB:** operates the national flood forecasting service and publishes daily water level bulletins and danger levels for monitored river stations. It is authoritative but not a village-level automatic alerting service.

- **Mobile/SMS flood alerts:** government and NGO-supported services broadcast warnings, usually based on official forecasts rather than a local sensor.
- **IoT water level monitoring:** many academic and hobby systems combine ESP8266/ESP32 with ultrasonic sensors and cloud dashboards. Typical gaps are limited alert staging, no role-based users and no structured software engineering documentation.
- **Messaging APIs:** the Telegram Bot API provides free programmatic notifications.

Gap addressed: a low-cost, locally deployable node that links sensing, server-side rules, history, role-based users and multi-channel alerts, designed and documented with a formal process.

6 SWOT Analysis

Strengths	**Weaknesses**
Real and important local problem; low cost; real-time alerts; simple technology (Java, ESP32); scalable to more nodes; strong SE structure	Prototype uses emulated water level; depends on WiFi and power; no forecasting; single-person effort; sensor accuracy not validated in field
Opportunities	**Threats**
Interest from BWDB/NGOs/disaster management; rising climate-related flood risk; high mobile penetration; cheap IoT hardware (ESP32, LoRa); open weather APIs; future research	False or missed alerts; sensor damage/theft in floods; network and power outage; overlap with existing official system; maintenance funding

Table 1: SWOT analysis of FloodGuard

Mitigation: use GSM/LoRa and local siren where internet is unavailable; battery/solar backup; averaging and outlier filtering of readings; position FloodGuard as a village-level complement to FFWC.

7 Requirements Specification

7.1 Functional requirements

ID	**Requirement**
FR1	The system shall receive water level readings from the ESP32 node via HTTP POST.
FR2	The system shall store each reading with a timestamp.
FR3	The system shall compute status (Safe/Warning/Danger) using the river's danger level.

ID	Requirement
FR4	The system shall generate an alert when the status changes to Warning or Danger.
FR5	The system shall display current level, status colour, progress bar and history graph on a dashboard.
FR6	The system shall show the alert list with time, village, level and message.
FR7	The Admin shall manage rivers, sensors, villages and users.
FR8	Residents and Volunteers shall be able to register and receive notifications.
FR9	The system shall support role-based login (Admin, Volunteer, Resident).
FR10	The ESP32 shall show status with green/yellow/red LEDs and a buzzer.
FR11	The system shall send Telegram and simulated SMS notifications, only on status change.

7.2 Non-functional requirements

ID	Requirement
NFR1	**Performance:** status update visible within 5 seconds of a reading.
NFR2	**Reliability:** local LED/buzzer alert works without internet; server must not crash if Telegram fails.
NFR3	**Usability:** simple dashboard usable on a mobile browser.
NFR4	**Security:** credentials and tokens not stored in source code; device key to validate readings; passwords hashed.
NFR5	**Maintainability:** layered design (Servlet–Service–DAO); danger level configurable.
NFR6	**Portability:** runs on any Servlet container (Jetty/Tomcat) and JDK 17+.
NFR7	**Cost:** prototype hardware under approximately BDT 1,100.

7.3 Input and output

Inputs: water level (sensor), river/village/user details (Admin, Residents), login credentials, simulation start. **Outputs:** dashboard (status, graph), alert list, LED/buzzer, Telegram and simulated SMS, stored history.

8 Feasibility Study

- **Technical:** ESP32, HC-SR04, Java, Maven, Jetty, Chart.js are mature and well documented; the developer has prior Servlet experience. Feasible.
- **Economic:** software is free; hardware is about BDT 750–1,100 or BDT 0 using Wokwi simulation. Feasible.
- **Operational:** simple colour-coded dashboard and messages are easy for volunteers and villagers.
- **Schedule:** 12 weeks with three increments is realistic.
- **Legal/ethical:** no sensitive personal data beyond name/phone; stored with restricted access.

9 Risk Analysis

#	Risk	Probability	Impact	Mitigation
1	Sensor noise / wrong reading	Medium	Medium	Average several samples; reject outliers
2	No internet at demo/site	High	High	Phone hotspot; LED/buzzer local alert; console fallback
3	Telegram token leak	Medium	High	Environment variables; `.gitignore`; revoke token
4	Hardware delay/failure	Medium	Medium	Wokwi simulation and Java simulator as backup
5	Fake data posted to server	Low	Medium	Device API key; range validation
6	Alert spam	Medium	Low	Notify only on status change
7	Schedule overrun	Medium	Medium	Incremental delivery; keep scope simple
8	Power outage in field	High	High	Battery/solar (future work)
9	Misuse as official forecast	Low	High	Clear disclaimer in UI and report

Table 4: Risk register

10 Cost and Resource Estimation

Item	Qty	Approx.
ESP32 development board	1	
HC-SR04 ultrasonic sensor	1	
LEDs (G/Y/R) + 220Ω resistors	3+3	
Active buzzer	1	
Breadboard + jumper wires	1 set	
Hardware total (optional)		**abo**
Software (IntelliJ Community, Maven, Jetty, MySQL, Git, Overleaf, Wokwi, Telegram Bot)		

Table 5: Estimated prototype cost (prices vary; verify locally)

Human resource: one developer, about 8–10 hours per week for 12 weeks. A field deployment would additionally need weatherproof enclosure, solar panel, battery and GSM/LoRa module, estimated separately per node.

11 Software Process Model: Incremental Model

11.1 Reason for selection

Requirements are reasonably clear, but the system has separable parts (data pipeline, dashboard, alerting). The Incremental model delivers a working product early and adds functionality in each increment, which reduces risk for a single-developer, time-boxed project.

11.2 Increments

Increment	Content	Output
Planning & Design	Problem analysis, requirements, schedule, UML	Proposal, diagrams
Increment 1	Servlet `/data`, River/Sensor/Reading/Alert classes, status logic, storage	Working data pipeline
Increment 2	Dashboard (colours, progress bar, Chart.js graph), alert list, simulation button	Live dashboard
Increment 3	Role-based login, notifications (console/Telegram), ESP32 + LEDs/buzzer, Wokwi	Complete prototype
Testing & Closure	JUnit and system tests, report, GitHub release	Final submission

Table 6: Increments

11.3 Weekly schedule

Week	Activity	Milestone
1–2	Idea finalisation, problem analysis, requirements, SWOT	M1
2–3	Process model, schedule, Gantt chart	M2
3–5	UML diagrams (DFD, Use case, Activity, ER, Class, Object)	M3
5–7	Increment 1: Servlet, model classes, storage	M4
7–9	Increment 2: dashboard, chart, alerts list	M5
9–10	Increment 3: login, Telegram, ESP32/Wokwi	M6
10–11	Testing and bug fixing	M7
11–12	Final report (LaTeX), GitHub, submission	M8

Table 7: Project schedule

11.4 Milestones and deliverables

ID	Milestone	Deliverable	Week
M1	Requirements complete	Requirements document, SWOT	2
M2	Plan approved	Schedule, Gantt chart, risk register	3
M3	Design complete	Six UML diagrams	5
M4	Increment 1 done	Data pipeline, source code	7
M5	Increment 2 done	Dashboard with graph and alerts	9
M6	Increment 3 done	Integrated system, ESP32 code, Wokwi project	10
M7	Testing done	Test cases and report	11
M8	Final submission	Proposal/report PDF, GitHub repository	12

Table 8: Milestones and deliverables

11.5 Gantt chart

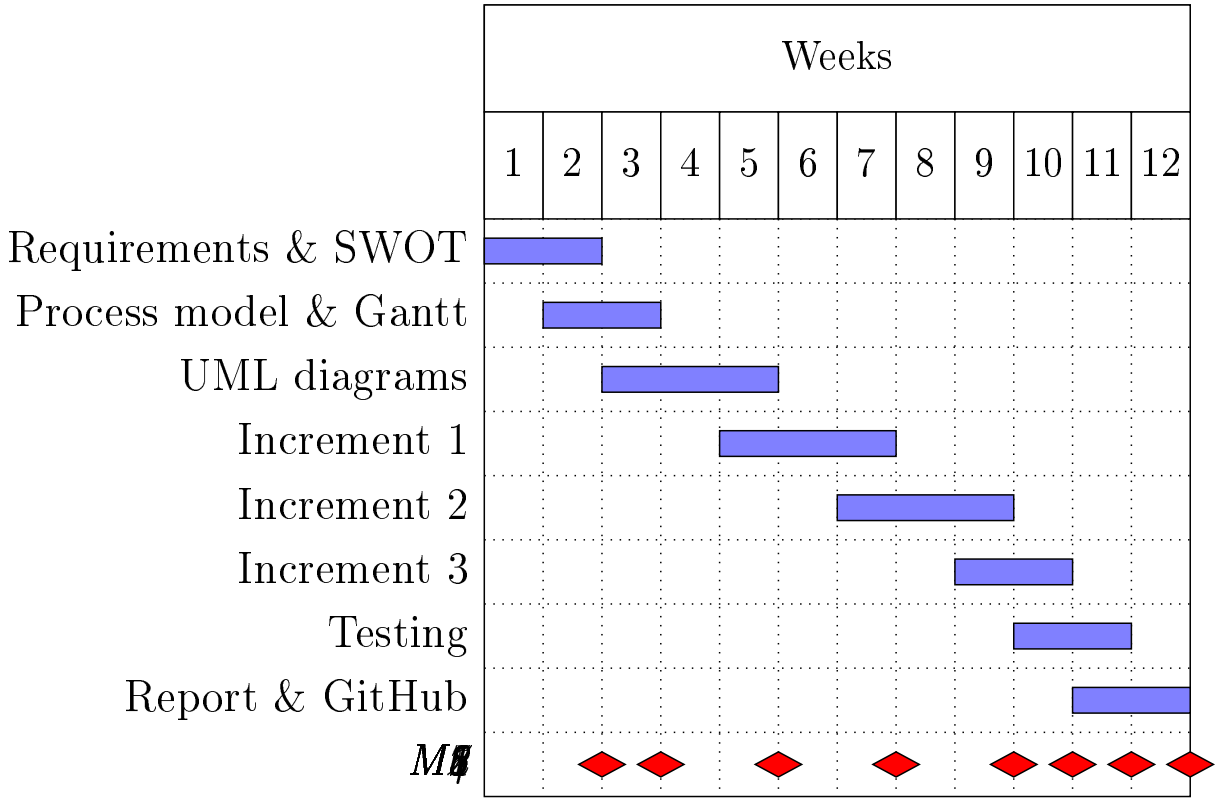


Figure 1: Gantt chart (12 weeks)

12 System Architecture and Technology Stack

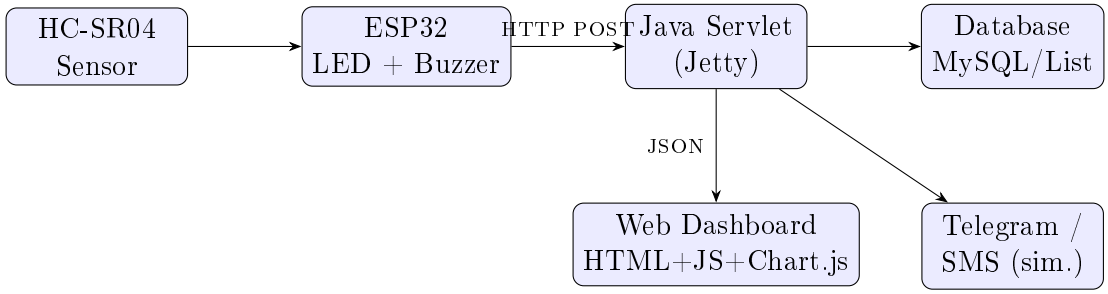


Figure 2: FloodGuard architecture

Layer	Technology
Device	ESP32, HC-SR04/JSN-SR04T, LEDs, buzzer (Arduino C++); Wokwi simulator
Backend	Java 17, Jakarta Servlet, DAO pattern, Maven, Jetty
Database	ArrayList (initial), MySQL/SQLite
Frontend	HTML, CSS, JavaScript, Chart.js
Notification	Console SMS simulation, Telegram Bot API
Tools	IntelliJ IDEA, Git/GitHub, Overleaf, PlantUML/TikZ, JUnit

Water level formula: with sensor height H above the river bed and measured distance d, level $= H - d$. **Status rule:** $p = \text{level}/\text{DangerLevel} \times 100$; Safe if $p < 80$, Warning if $80 \leq p < 100$, Danger if $p \geq 100$. The danger level is taken from FFWC station data and is configurable.

13 UML Diagrams

13.1 Data Flow Diagram (Level 0 and Level 1)

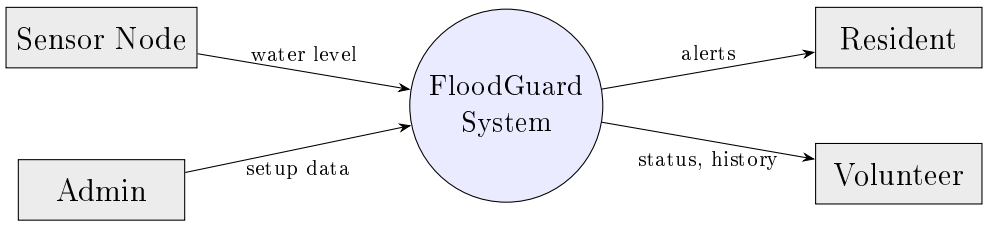


Figure 3: DFD Level 0 (context diagram)

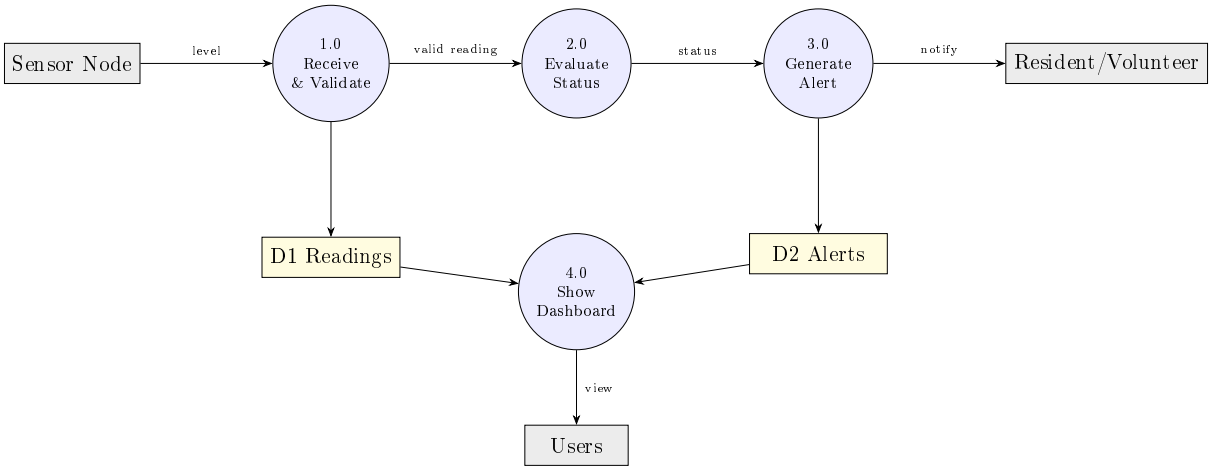


Figure 4: DFD Level 1

13.2 Use Case Diagram

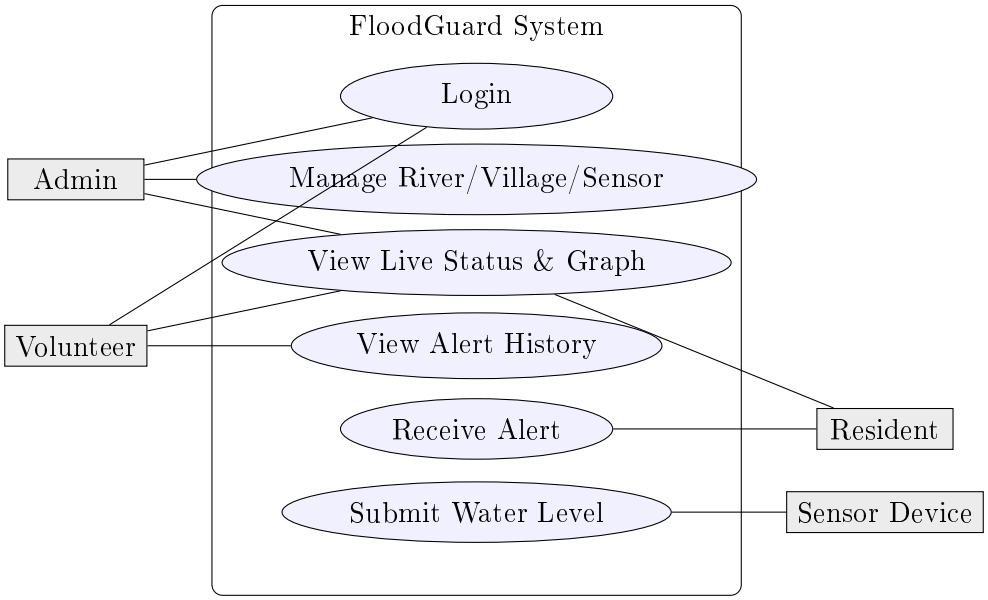


Figure 5: Use case diagram

13.3 Activity Diagram

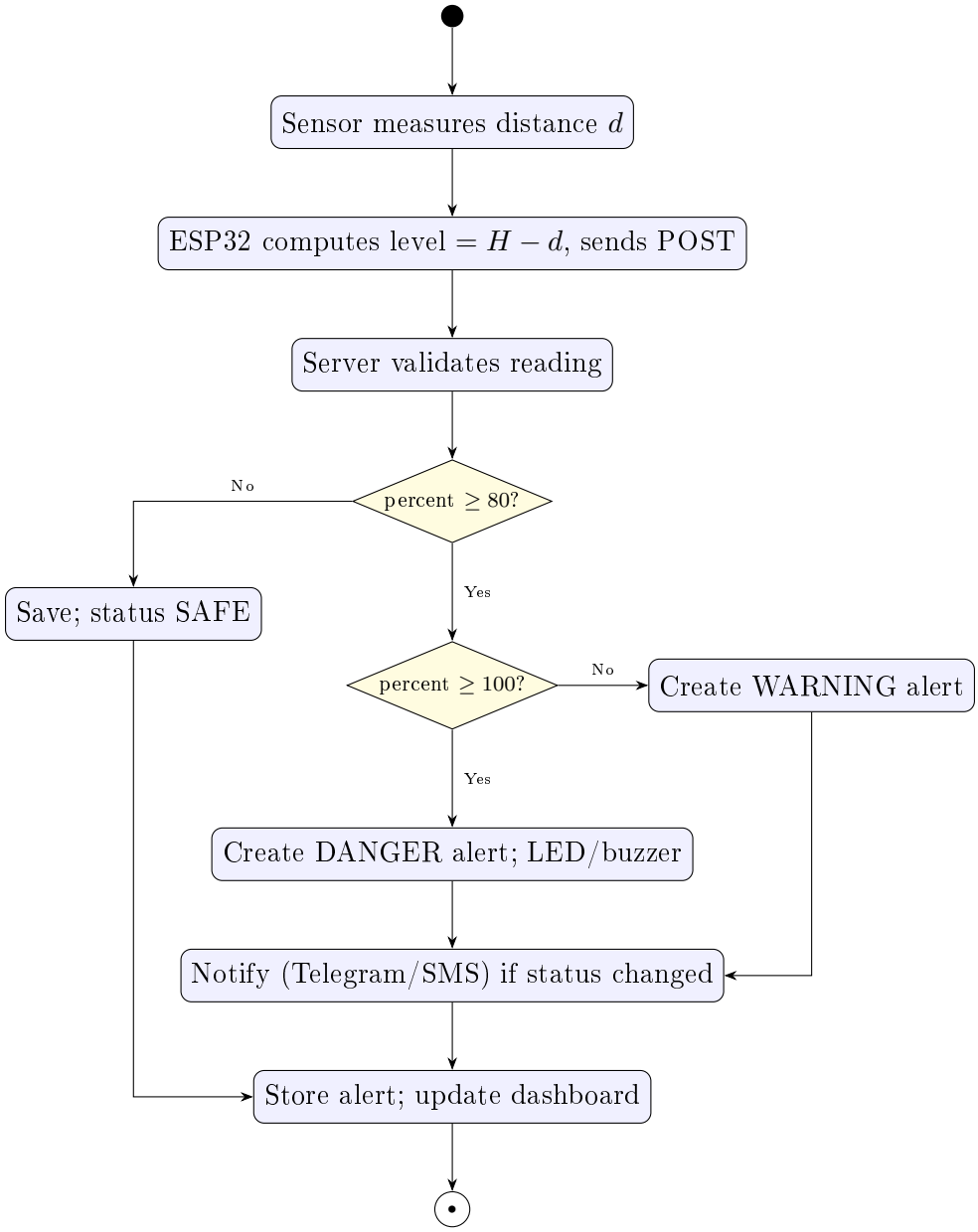


Figure 6: Activity diagram: processing a reading

13.4 Entity Relationship Diagram

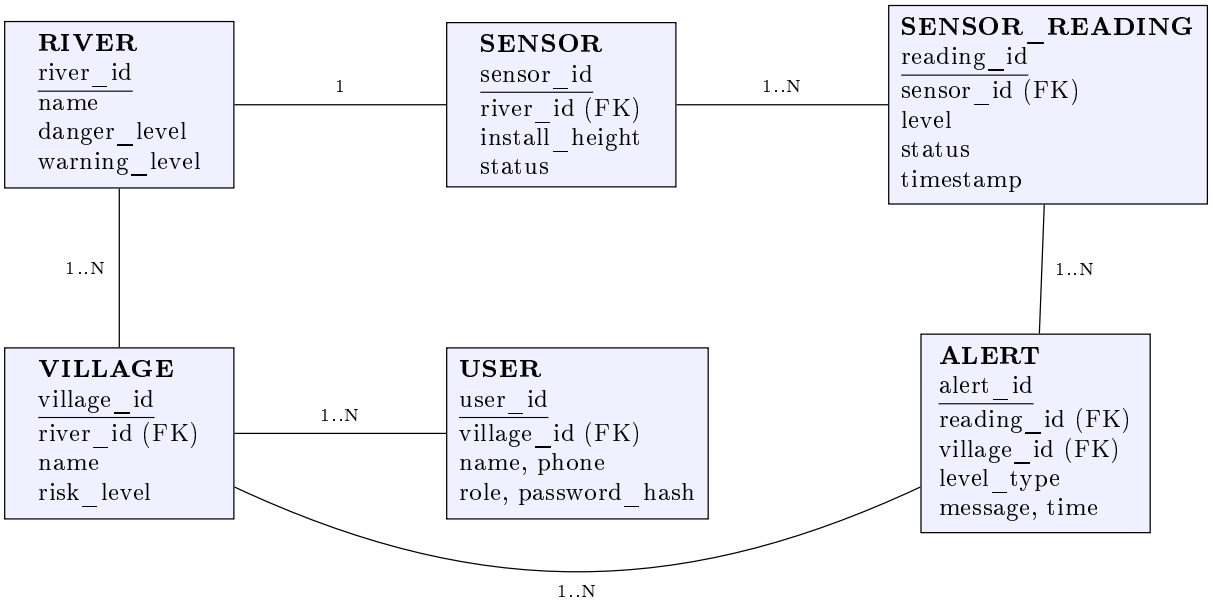


Figure 7: ER diagram (crow's-foot cardinalities shown as 1 and 1..N)

13.5 Class Diagram

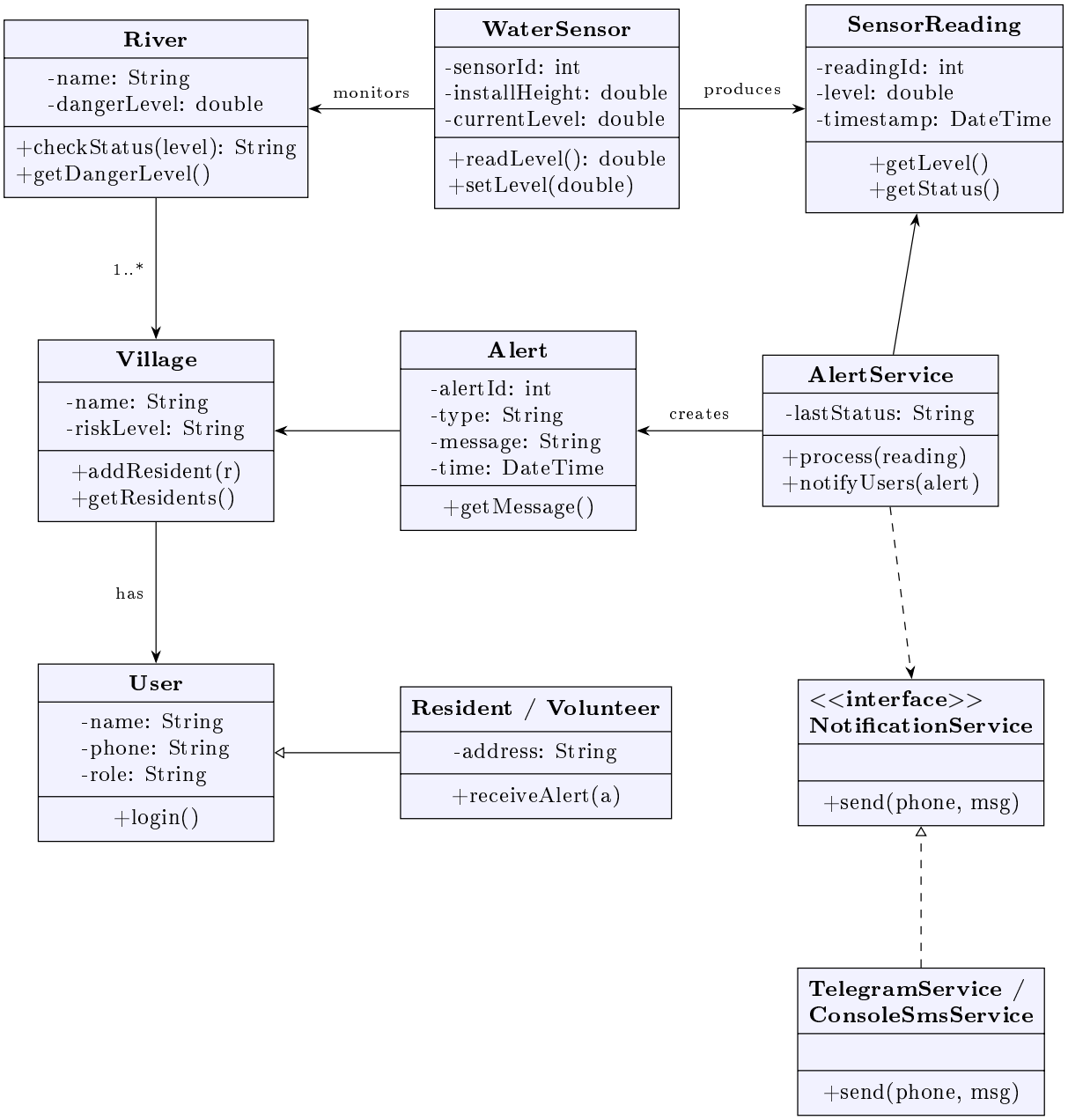


Figure 8: Class diagram

13.6 Object Diagram

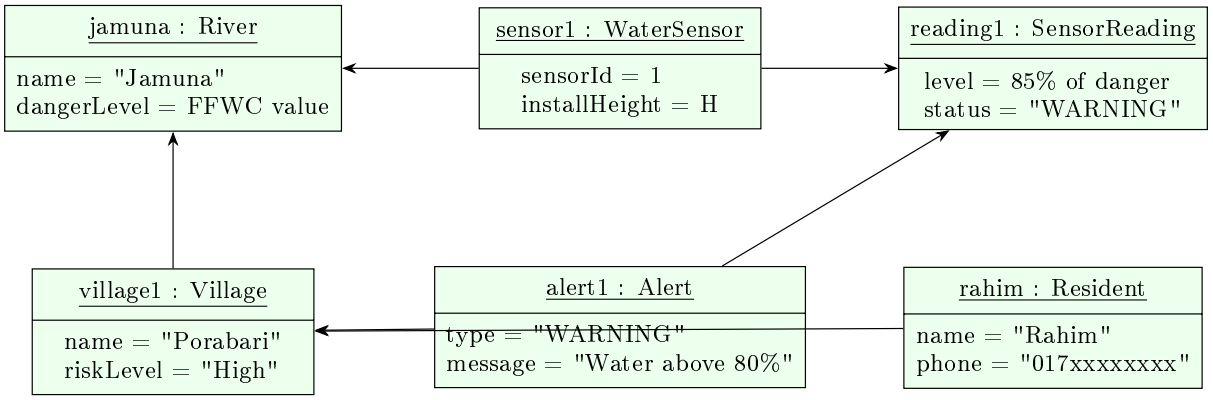


Figure 9: Object diagram (sample snapshot)

14 Simple Interface Implementation Plan

The interface is a browser dashboard served by a Java Servlet. At least five classes from the class diagram are implemented: `River`, `WaterSensor`, `SensorReading`, `Alert`, `AlertService` (plus `NotificationService` implementations).

Listing 1: Status logic in River (core of the system)

```
public class River {
    private String name;
    private double dangerLevel;
    public River(String name, double dangerLevel) {
        this.name = name; this.dangerLevel = dangerLevel;
    }
    public String checkStatus(double level) {
        double p = level / dangerLevel * 100;
        if (p >= 100) return "DANGER";
        if (p >= 80)  return "WARNING";
        return "SAFE";
    }
}
```

Dashboard shows: river name, current level, status colour (green/yellow/red), progress bar, level-versus-time graph, alert list, and a "Start Simulation" button.

ESP32 firmware (summary): read HC-SR04, compute $H - d$, show status on green/yellow/red LEDs and buzzer, and send the level to `/data` using HTTP POST.

15 Testing Plan

ID	Input	Expected	Type
T1	50% of danger level	SAFE, green	Unit (JUnit)
T2	85% of danger level	WARNING, yellow, alert	Unit
T3	105% of danger level	DANGER, red, alert, buzzer	Unit/System
T4	Same status repeated	No duplicate notification	Unit
T5	Telegram unreachable	Server continues, console fallback	System
T6	Invalid/negative reading	Rejected	Unit

Table 9: Sample test cases

16 Conclusion and Future Work

FloodGuard demonstrates a practical, low-cost IoT approach to village-level flood early warning, developed with a structured software engineering process (Incremental model), complete UML modelling and a realistic schedule. The prototype shows the full chain from sensing to alerting while being transparent about what is emulated. Future work includes LoRa/GSM connectivity, solar power, multiple nodes, a mobile app, integration of rainfall and upstream data, and machine learning based forecasting.